

# Backend Engineering Technical Exercise

KAD Global Agency | Confidential

---

|                        |                                                                                                                                                                                                                                                                                                                                      |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Role:</b>           | Backend Engineer                                                                                                                                                                                                                                                                                                                     |
| <b>Technology:</b>     | Python 3.10+ (required). HTTP framework of your choice: <b>Flask + flask-socketio is the closest match to our production stack</b> ; FastAPI + python-socketio is also accepted; Starlette and others are welcome if you prefer. MQTT via <code>paho-mqtt</code> or equivalent. Database: your choice (SQLite, PostgreSQL, MongoDB). |
| <b>Estimated Time:</b> | 5–7 hours                                                                                                                                                                                                                                                                                                                            |
| <b>Submission:</b>     | Source code (Git repository or zip) + README + run instructions + sample <code>verify.py</code> output                                                                                                                                                                                                                               |
| <b>Send to:</b>        | muhammed.khan@truxel.ai, monica@kad.agency                                                                                                                                                                                                                                                                                           |

## 1. Objective

Build a small, correct, concurrent-safe backend service that exposes an HTTP API, coordinates with an external worker over an MQTT broker, and pushes real-time state updates to clients over a WebSocket. The service manages a simple domain object (a "job") with a strict state machine, per-user authorization, idempotent submission, and an atomic conflict-detection rule.

We are evaluating **HTTP/REST API design, state-machine modeling, concurrency safety** (atomic operations under contention), **authentication & authorization, idempotent request handling, pub/sub integration** (MQTT), **real-time push** (WebSocket), **persistence + transactions, test discipline**, and **clarity of documentation**. We are not evaluating UI or production deployment.

## 2. Scenario

An operator submits a **job** targeting a physical **asset** (a battery, a charger, etc.) for a future time window. Your service validates the job, persists it, and dispatches it to a worker by publishing on an MQTT topic. The worker processes the job asynchronously and publishes state transitions back on a results topic. Your service consumes those transitions, updates the job state, and pushes updates to connected operators in real time over a WebSocket.

The system you are building sits between three things:

- **Operators** — humans (or programs) calling your HTTP API. They authenticate with a Bearer token and may only see / submit their own jobs.
- **The MQTT broker** — carries job dispatches out to the worker and results back. We ship a Mosquitto broker plus the worker.

- **WebSocket clients** — the operator's UI (we don't build this). Your service emits `job_update` events scoped to each operator's own jobs.

### 3. Provided Components

The exercise ships with everything you need to develop against:

```
be-exercise-bundle/
├── backend-engineer-exercise.pdf    # this document
├── README.md                      # candidate-facing guide
├── docker-compose.yml              # Mosquitto + worker
├── mosquitto.conf                  # broker config (anonymous, port 1883)
├── worker/                         # worker stub: source + Dockerfile
├── data/
│   └── seed.json                  # users (with API tokens) + assets
└── verify.py                      # self-assessment test harness
```

#### 3.1 Provided MQTT broker

Mosquitto 2.x on `localhost:1883`, anonymous access. Start with:

```
docker compose up -d
```

#### 3.2 Provided worker

The worker subscribes to `jobs/dispatch` and publishes to `jobs/result`. It is deterministic so your tests are reproducible:

- Receives a dispatch payload. Immediately publishes `{"state": "RUNNING"}`.
- Sleeps ~1 second (configurable via `WORKER_PROCESSING_SECONDS`).
- If `asset_id == "asset-fault"` → publishes `{"state": "FAILED", "error_message": "..."}.`
- Otherwise → publishes `{"state": "COMPLETED"}`.

The worker **always emits `RUNNING` before any terminal state**. The `PENDING` → `COMPLETED` / `PENDING` → `FAILED` transitions listed in §4.3 are permitted by the spec but never produced by the shipped worker.

Source is in `worker/worker.py` for reference; you do not need to modify it.

#### 3.3 Seed data ( `data/seed.json` )

Three users, each with a static Bearer token:

| user_id  | name           | token                  |
|----------|----------------|------------------------|
| user-001 | Alice Operator | tok_alice_a1b2c3d4e5f6 |
| user-002 | Bob Operator   | tok_bob_b2c3d4e5f6a1   |
| user-003 | Carol Operator | tok_carol_c3d4e5f6a1b2 |

Nine assets, including `asset-fault` which the worker treats as a forced-failure case. The full list and operation enum are in `seed.json`.

### **3.4 `verify.py` — the self-assessment harness**

A ready-to-run integration test script. After your service is running, execute `python3 verify.py` and it reports PASS / FAIL across roughly 23 checks covering every functional requirement below. If everything passes, you can be confident your implementation meets the spec.

## 4. Functional Requirements

### 4.1 HTTP REST API

Expose these endpoints on a port of your choice (default: 8000). All API endpoints under `/api/` require Bearer-token authentication (see §4.2). `/health` does not require auth.

| Method & path                   | Description                                                                                                               |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| GET <code>/health</code>        | Liveness check. Returns 200 with at minimum <code>{"status":"ok"}</code> . Including MQTT connectivity status is welcome. |
| POST <code>/api/jobs</code>     | Submit a new job. <b>Requires header</b> <code>Idempotency-Key</code> (non-empty, ≤ 100 chars, scoped per user).          |
| GET <code>/api/jobs/{id}</code> | Read a single job. Returns 404 if the job does not exist or it belongs to another user.                                   |
| GET <code>/api/jobs</code>      | List the caller's jobs (only). 200 always; empty list if none.                                                            |

**Request body for POST `/api/jobs` :**

```
{
  "asset_id": "asset-001",
  "operation": "charge",
  "start_time": "2026-05-16T14:00:00.000Z",
  "end_time": "2026-05-16T16:00:00.000Z"
}
```

**Response body for job objects** (used by all three job endpoints; field order is not significant):

```
{
  "id": "uuid",
  "asset_id": "asset-001",
  "operation": "charge",
  "start_time": "2026-05-16T14:00:00.000Z",
  "end_time": "2026-05-16T16:00:00.000Z",
  "state": "PENDING", // see §4.3
  "error_message": null,
  "created_at": "2026-05-15T10:00:00.000Z",
  "updated_at": "2026-05-15T10:00:00.000Z"
}
```

**HTTP status code matrix:**

| Status      | When                                                                                   |
|-------------|----------------------------------------------------------------------------------------|
| 201 Created | POST <code>/api/jobs</code> succeeded. Also returned on idempotent replay (same body). |
| 200 OK      | GET endpoints returning data; health check.                                            |

|                          |                                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------------|
| 400 Bad Request          | Malformed request (e.g. invalid JSON, bad ISO-8601 timestamp, missing Idempotency-Key ).                   |
| 401 Unauthorized         | Missing or invalid Authorization header.                                                                   |
| 404 Not Found            | GET on a job that does not exist or belongs to another user. (Returning 403 is also acceptable.)           |
| 409 Conflict             | Submitted job overlaps an existing PENDING or RUNNING job on the same asset (see §4.5).                    |
| 422 Unprocessable Entity | Body parses but violates a business rule (unknown asset, past start_time , bad duration, etc. — see §4.4). |

## 4.2 Authentication & authorization

- Every `/api/*` endpoint requires header `Authorization: Bearer <token>` .
- Valid tokens are the ones listed in `data/seed.json` . Tokens are static fixtures for the exercise — treat them like test secrets, not production credentials.
- Each request maps to a single `user_id` . All read endpoints must scope results to that user only. A user attempting to read another user's job must receive 404 (or 403 if you prefer; not 200).

## 4.3 State machine

A job is always in exactly one of these four states:

| State     | Meaning                                                             | Entered by                                                                 |
|-----------|---------------------------------------------------------------------|----------------------------------------------------------------------------|
| PENDING   | Accepted by the API, waiting for the worker to pick up.             | Initial state on successful <code>POST /api/jobs</code> .                  |
| RUNNING   | Worker is processing.                                               | MQTT result message with <code>{"state":"RUNNING"}</code> .                |
| COMPLETED | Worker finished successfully.                                       | MQTT result with <code>{"state":"COMPLETED"}</code> .                      |
| FAILED    | Worker reported a failure. <code>error_message</code> is populated. | MQTT result with <code>{"state":"FAILED", "error_message": "..."} .</code> |

**Allowed transitions** (any other inbound transition must be ignored / not persisted):

| From      | To                         |
|-----------|----------------------------|
| PENDING   | RUNNING, COMPLETED, FAILED |
| RUNNING   | COMPLETED, FAILED          |
| COMPLETED | — (terminal)               |
| FAILED    | — (terminal)               |

State transitions are driven only by MQTT messages from the worker. The HTTP API does not expose a way to mutate state directly.

#### 4.4 Business-rule validation

On `POST /api/jobs`, enforce all of:

- `asset_id` must exist in `seed.json` → else 422.
- `start_time` and `end_time` must be ISO-8601 UTC with `Z` or `+00:00` → else 400.
- `start_time < end_time` → else 422.
- `start_time ≥ now` (must be in the future) → else 422.
- Duration  $\geq$  15 minutes and  $\leq$  4 hours → else 422.
- `operation` is a non-empty string up to 64 chars. No enum enforcement required; `seed.json` lists suggested values.
- Each user may have at most **10 active** (PENDING + RUNNING) jobs → else 422.

#### 4.5 Atomic overlap check (concurrency requirement)

Two active jobs (PENDING or RUNNING) on the **same asset** must not have overlapping time windows. Windows are half-open `[start_time, end_time)`; two windows `[a, b)` and `[c, d)` overlap iff `a < d AND c < b`.

This check must be **atomic against concurrent submissions**. If ten clients submit overlapping windows on the same asset simultaneously, exactly one must return 201 and the rest must return 409. A naive "select-then-insert" outside a transaction will not pass `verify.py`'s concurrency test.

You may use database transactions with appropriate isolation, row-level locks, unique constraints, or any other primitive that gives you a single atomic write-with-pre-condition. Whichever you pick, defend it briefly in the README.

## 4.6 Idempotency

- `POST /api/jobs` requires header `Idempotency-Key` (non-empty,  $\leq 100$  chars). Missing  $\rightarrow$  400.
- The key is scoped **per user**: a key collision between different users creates two independent jobs.
- If the same `(user_id, Idempotency-Key)` tuple is seen a second time, you must return the *same* job (same `id`) with status 201 and not create a new row.
- You do not need to enforce that the second request's body matches the first; you may treat the first body as canonical.

## 4.7 MQTT integration

Connect to the broker at `localhost:1883` with QoS 0 (at-most-once is sufficient for this exercise).

| Topic                      | Direction                         | Payload                                                                        |
|----------------------------|-----------------------------------|--------------------------------------------------------------------------------|
| <code>jobs/dispatch</code> | your service $\rightarrow$ worker | <code>{job_id, asset_id, start_time, end_time, operation}</code> (all strings) |
| <code>jobs/result</code>   | worker $\rightarrow$ your service | <code>{job_id, state, timestamp, error_message?}</code>                        |

Publish a dispatch **after** you have persisted the job (don't dispatch a job that didn't successfully commit). On a result message:

- If the `job_id` is unknown, log and ignore.
- If the proposed transition is invalid (e.g. `COMPLETED  $\rightarrow$  RUNNING`), log and ignore.
- Otherwise persist the new state (and `error_message` if present), update `updated_at`, and emit the WebSocket event (§4.8).

## 4.8 WebSocket real-time updates

Use the **Socket.IO protocol** (the same one used by `python-socketio` and `flask-socketio`). **Mount the Socket.IO server at the default path `/socket.io/`** — `verify.py` connects there and will fail if you change the mount path.

- **Auth:** the client connects with the token as a query-string parameter, e.g. `/socket.io/?token=<bearer>`. Reject the connection if the token is missing or invalid.
- **Scoping:** immediately after a successful connection, put the socket into a per-user "room" (Socket.IO room name is your choice; `user:<user_id>` is a sensible default).
- **Event:** when a job changes state, emit a `job_update` event *only* to that job's owner's room. Payload: `{"job_id": "...", "state": "RUNNING", "timestamp": "..."}.`
- A user must **not** receive events for jobs they don't own. `verify.py` tests this directly.

## 4.9 Idempotent re-runs & determinism

- Re-running `POST /api/jobs` with the same Idempotency-Key produces the same response (per §4.6).
- Re-running `verify.py` twice in a row must produce the same PASS / FAIL outcome (start with a fresh DB; `verify.py` uses fresh asset windows for each scenario).

#### 4.10 What you are *not* asked to do

- No frontend. No HTML, no styling, no UI.
- No cancel endpoint. State transitions only via MQTT from the worker.
- No rate limiting, no metrics endpoint, no audit log requirement.
- No production-grade auth: static Bearer tokens are sufficient. JWT is not required.
- No webhooks, no HMAC signing. The exercise does not use HTTP webhooks at all.
- No backfill or replay mode.



## 5. Pipeline Entry Point

Provide a single command to start your service. Recommended:

```
make setup      # install dependencies
make run        # start your service on port 8000
make test       # run your unit / integration tests
```

Before you run your service, start the broker + worker:

```
docker compose up -d      # boots Mosquitto + worker
```

Verify your implementation with:

```
python3 verify.py          # runs the full self-assessment harness
```

## 6. Technical Expectations

- Python 3.10+. Pin dependencies ( `requirements.txt` , `pyproject.toml` , or `uv.lock` ).
- HTTP framework: **FastAPI**, Flask, or Starlette. FastAPI is recommended (clean async, easy ASGI mount of Socket.IO).
- WebSocket: `python-socketio` with an ASGI adapter (or equivalent).
- MQTT: `paho-mqtt` 2.x.
- Database: SQLite, PostgreSQL, or MongoDB — your choice. Defend the choice in the README. SQLite with `BEGIN IMMEDIATE` is a perfectly valid option for this exercise.
- Modular package layout: separate ingestion, persistence, validation, MQTT handling, WebSocket handling. Single-file solutions are acceptable but harder to review.
- Structured logging (one JSON object per line, or equivalent) at request boundaries.
- Configuration via environment variables; no hard-coded paths or secrets.
- **Tests — at least three:**
  - Idempotency: same Idempotency-Key twice returns the same job.
  - Atomic concurrency: parallel overlapping submissions return exactly one 201.
  - End-to-end MQTT round-trip: submit → worker → state becomes COMPLETED.

Tests may use a small embedded MQTT broker, mock the broker, or hit the real shipped one — your choice.

- The full `verify.py` harness must pass against your running service.

## 7. Documentation (Required)

Include a `README.md` covering, concisely:

- **How to run** — setup, the full-stack startup command, how to run tests, how to run `verify.py` .
- **Architecture** — one or two paragraphs describing the layers and the flow of a job through them.
- **Database choice** — what you picked and why.

- **Concurrency strategy** — how you make the overlap check atomic.
- **Idempotency strategy** — where the key is stored and how the second request finds the first.
- **MQTT & Socket.IO threading** — how the synchronous MQTT loop talks to the asyncio Socket.IO server. (This is a real gotcha.)
- **Trade-offs you took** — what you cut for the time budget, what you would build next.

Aim for two to four pages. We value concision and honest reasoning over completeness.

## 8. Submission

- Source code as a Git repository (preferred) or zip.
- `README.md` per §7.
- A `Makefile` (or equivalent) with `make setup`, `make run`, `make test`.
- Paste the output of a successful `verify.py` run into the README or a separate `verify-output.txt`.

## 9. Evaluation Criteria

| Criteria                                  | What we look for                                                                                                             |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Correctness</b>                        | All scenarios in <code>verify.py</code> pass against your running service.                                                   |
| <b>Concurrency safety</b>                 | The 10-parallel overlap test in <code>verify.py</code> consistently returns exactly 1× 201 and 9× 409, every run.            |
| <b>State-machine discipline</b>           | Invalid transitions are rejected silently (logged, not crashed). MQTT messages for unknown jobs are tolerated.               |
| <b>Authentication &amp; authorization</b> | Tokens enforced on every <code>/api</code> path; cross-user reads return 404 (or 403); WebSocket events are scoped per user. |
| <b>Idempotent submission</b>              | Same Idempotency-Key returns the same job; second POST does not insert a second row.                                         |
| <b>MQTT integration</b>                   | Service publishes dispatches only after persistence; processes results without losing the loop on a bad message.             |
| <b>WebSocket integration</b>              | Token-auth on connect, per-user scoping on emit, transitions delivered in order.                                             |
| <b>Code structure</b>                     | Clear separation of concerns; readable; small testable units.                                                                |
| <b>Testing</b>                            | The three required tests exist, pass, and exercise the spec rather than smoke-tests.                                         |
| <b>Documentation</b>                      | README answers the required questions with honest trade-off discussion.                                                      |

Good luck. We look forward to reviewing your submission.

— KAD Global Agency, Backend & Platform Team